Teaching Coding Agents to Master *Spreadsheets*

Nuno Campos

AI Engineering London — April 2026

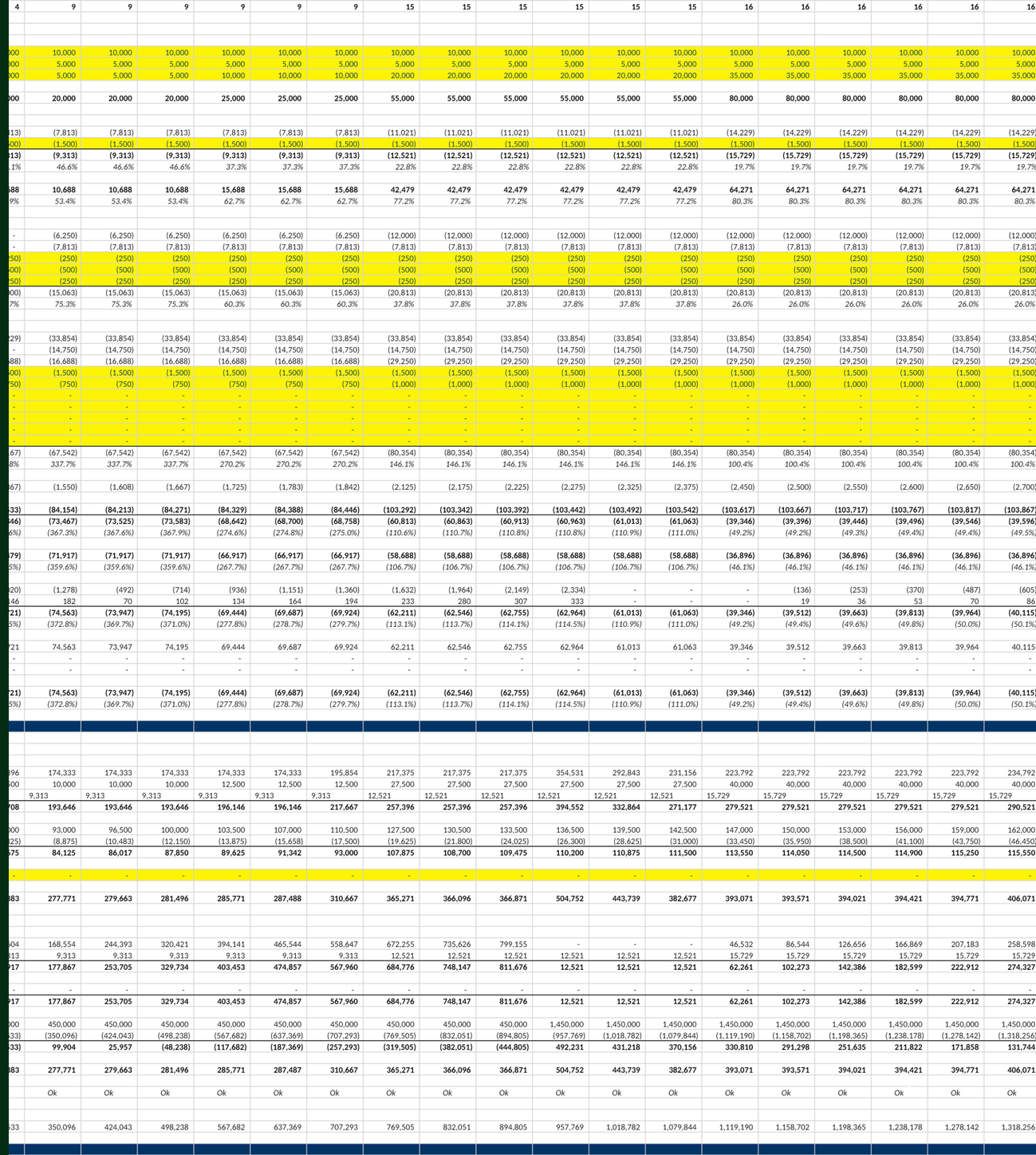
50% → 92%

- 4 months, multiple architectures, and many dead ends
- What mattered most: replacing 15 discrete tools with one REPL

The problem

A human sees: revenue table, assumptions, P&L summary

An LLM sees: 10,000 cell values, `=SUMPRODUCT((B$3:B$50="NEW")*(G$3:G$50))`, formatting metadata



Attempt 1: Turn the spreadsheet into a database

- Excel → SQLite (every cell, formula, merged region, color block)
- Agent: LangGraph + GPT-5, tools for SQL query / write cell / insert row

Result: 50% — then 73% after fixing a one-character extraction bug

Three specialized agents:

1. Block discovery (low effort) — identifies workbook structure
2. Edit agent (high effort) — 5-step process: disambiguate, define end state, plan, execute, verify
3. Question agent (read-only) — answers questions

Key finding: "Define the end state before you touch a cell" was the most impactful prompt instruction

The dead ends

Representation	Why it failed
TSV views	Lost formatting and structural context
SQL views	Flat tables didn't capture visual structure
HTML tables	Too verbose, consumed too many tokens
DOT graphs	Useful for analysis, not for interaction
XML/XSLT	LLM struggled with the syntax

None of them worked as a general-purpose representation — but two informed what came next.

We replaced all 15 tools with a persistent Node.js REPL and a spreadsheet API.

```
Agent Loop --(JavaScript code)--> Node.js REPL --(JSON-RPC)--> .NET engine
                                      |                            |
                                  Variables persist            Workbook state
                                  across calls                 persists
```

Before vs. After

Before (10-15 tool calls):

```
Tool call 1: list_sheets()              -> ["Summary", "Data", "Inputs"]
Tool call 2: read_range("Summary!A1:D10") -> cell values
Tool call 3: find_cells("Revenue")    -> [matches]
Tool call 4: read_cell("Summary!C10") -> "$1,234,567"
...
```

After (1 tool call):

```
const sheets = await xlsx.listSheets(wb);
const summary = await xlsx.readRangeTsv(wb, `${sheets[0].name}!A1:D10`);
const revenue = await xlsx.findCells(wb, "Revenue", { context: 1 });
console.log(sheets, summary, revenue);
```

Code mode vs. REPL

Code mode — the agent writes scripts instead of making tool calls. Operations compose naturally. 50-line scripts are common.

REPL = **code mode** + **persistent state** — variables survive across calls. The agent writes shorter scripts, reasons between them, builds understanding incrementally.

```
// Code mode: one long script, print everything at the end
// REPL: explore, reason, continue
const revenue = await xlsx.findCells(wb, "Revenue", { context: 1 });
console.log(revenue);
// → agent reasons about the output, then writes the next script
```

Result: small accuracy gain on harder tasks — the agent can course-correct mid-exploration.

The results

Date	Pass rate	Tasks
Nov 30	**74%**	104
Dec 11	**88%**	167
Dec 14	**92.1%**	165

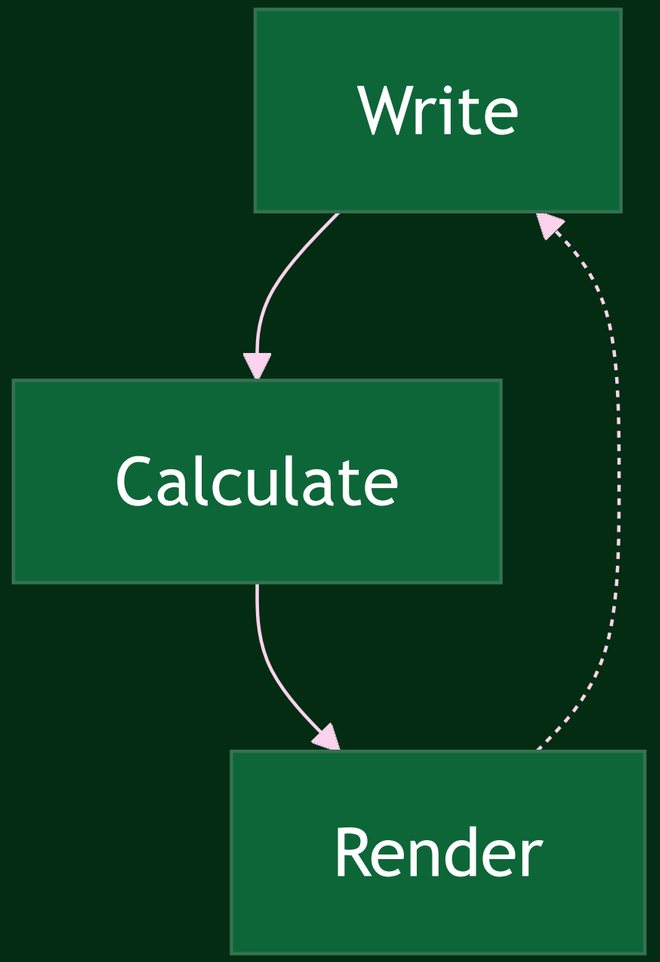
Zero timeouts. 50-second average runtime.

18 points in two weeks from compounding small gains:
better search, new API functions, improved docs, backend bug fixes

The verification loop

The formula engine and renderer close a loop the agent uses to check its own work.

This held across three successive frontier model releases. Each new model used the same loop more effectively.



The **REPL** is an interface — the best one today, because coding is where models are strongest.

The **engines** — formula calculation, rendering, linting — are the durable part. They're what close the verification loop, and they compound with each new model.

If agents become as capable at computer use as they are at coding, the interface might change. The engines won't.

The test that contradicted our thesis

We'd built a separate set of CLI commands — `find`, `calc`, `render`, `lint` — designed as standalone tools any agent could call.

We tested them against plain openpyxl on 20 QnA tasks. Same model, same runner.

We expected ours to win. It had a formula engine, semantic linting, visual rendering. openpyxl can't even recalculate formulas.

openpyxl won

	openpyxl	Witan CLI
Pass rate	**85%**	**70%**
Avg tool calls	21	42

The default approach won by 15 points.

Why it lost

Every CLI command was a separate process — startup, shell parsing, .NET engine initialization. That overhead compounded:

1. **A recalculation bug** made each command take 50-130 seconds. 20+ commands per task meant timeouts.
2. **Shell escaping** across three layers (SDK, zsh, .NET parser) broke on sheet names with spaces.
3. **Our documentation** had the quoting examples backwards. The agent followed our wrong instructions faithfully.

openpyxl is an in-process library. No subprocesses, no shell, no infrastructure to go wrong.

This test showed why the REPL worked where the CLI didn't: **a single invocation runs an entire script without spawning a process per operation.**

Two products emerged:

1. **exec** — the REPL, externalized as a CLI command any agent can use
2. **verify** — render, calc, lint as a lightweight add-on for agents that already have their own spreadsheet tools

Domain knowledge outlived every tool

- We changed tools four times in four months
- The financial domain knowledge improved results on **every one of them**

Structured as a composable prompt component we call the "Financial Expert Mindset":

- How to interpret margins, profitability, revenue cascades
- Model type recognition (DCF, LBO, three-statement)
- Communication conventions ($1.2M not $1,234,567.89)
- "Never calculate in your head what the spreadsheet can calculate for you"

It was the most reused component in the system.

Evaluation was half the work

We started with LLM-as-judge. We couldn't tell if a score change was the agent improving or the evaluator being flaky.

We replaced it with deterministic comparison wherever possible — programmatic checks for values, layout, and formatting. LLM grading only for genuinely subjective text answers.

568 commits. Nearly as complex as the agent itself.

Infrastructure bugs look like reasoning failures

What it looked like	What it actually was
Agent can't find data	One-character extraction bug (50% → 73%)
Agent uses wrong quoting	SKILL.md had backwards examples
Agent times out constantly	Per-cell recalculation query instead of batch
Agent retries endlessly	API returning empty results intermittently

When the agent seems confused, check the plumbing first.

What generalizes

1. **Many small sequential tool calls = a bad scripting language.** Give the agent a real one.
2. **Build verification engines.** Formula calc, rendering, and linting close a feedback loop that compounds with model capability rather than being replaced by it.
3. **Interfaces are ephemeral, engines are durable.** The REPL works because coding is today's strongest model skill. That may not last.
4. **Structured reasoning > better tools.** "Define the end state before you act" caught more errors than any tool improvement.
5. **Domain knowledge outlasts tools.** Four backends in four months. The domain knowledge improved results on all of them.
6. **Match evaluation to output type. Check the plumbing first.** Use deterministic comparison for objective outputs, LLM grading for subjective ones. Agent "confusion" is usually infrastructure.

50% → 92%. Four months.

We spent four months trying to constrain the agent into tighter interactions. It worked better when we gave it a programming environment and got out of the way.

Research log: github.com/witanlabs/research-log


WITAN